

Scraped Content

Programming > Python

Source: Python Docs

URL: <https://docs.python.org/3/tutorial/introduction.html>

- 3. An Informal Introduction to Python 3. An Informal Introduction to Python In the following examples, input and output are distinguished by the presence or absence of prompts (and): to repeat the example, you must type everything after the prompt, when the prompt appears; lines that do not begin with a prompt are output from the interpreter. Note that a secondary prompt on a line by itself in an example means you must type a blank line; this is used to end a multi-line command. You can use the Copy button (it appears in the upper-right corner when hovering over or tapping a code example), which strips prompts and omits output, to copy and paste the input lines into your interpreter. Many of the examples in this manual, even those entered at the interactive prompt, include comments.
- Comments in Python start with the hash character, #, and extend to the end of the physical line. A comment may appear at the start of a line or following whitespace or code, but not within a string literal. A hash character within a string literal is just a hash character. Since comments are to clarify code and are not interpreted by Python, they may be omitted when typing in examples.
- 3.1. Using Python as a Calculator Let's try some simple Python commands. Start the interpreter and wait for the primary prompt, >. (It shouldn't take long.) The interpreter acts as a simple calculator: you can type an expression into it and it will write the value.
- Expression syntax is straightforward: the operators +, -, and * can be used to perform arithmetic; parentheses (()) can be used for grouping. For example: The integer numbers (e.g. 2, 4, 20) have type int, the ones with a fractional part (e.g. 5.0, 1.6) have type float. We will see more about numeric types later in the tutorial. Division (/) always returns a float.
- To do floor division and get an integer result you can use the // operator; to calculate the remainder you can use the % operator. With Python, it is possible to use the ** operator to calculate powers. The equal sign (=) is used to assign a value to a variable. Afterwards, no result is displayed before the next interactive prompt: If a variable is not defined (assigned a value), trying to use it will give you an error: There is full support for floating point; operators with mixed type operands convert the integer operand to floating point: In interactive mode, the last printed expression is assigned to the variable _. This means that when you are using Python as a desk calculator, it is somewhat easier to continue calculations, for example: This variable should be treated as read-only by the user. Don't explicitly assign a value to it - you would create an independent local variable with the same name masking the built-in variable with its magic behavior.
- In addition to int and float, Python supports other types of numbers, such as Decimal and Fraction. Python also has built-in support for complex numbers, and uses the j suffix to indicate the imaginary part (e.g. 35j). Python can manipulate text (represented by type str, so-called "strings") as well as numbers. This includes characters !, words rabbit, names Paris, sentences Got your back., etc.
- Yay! :) They can be enclosed in single quotes (') or double quotes (") with the same result. To quote a quote, we need to escape it, by preceding it with \. Alternatively, we

can use the other type of quotation marks: In the Python shell, the string definition and output string can look different.

- The `print()` function produces a more readable output, by omitting the enclosing quotes and by printing escaped and special characters: If you don't want characters prefaced by `\` to be interpreted as special characters, you can use raw strings by adding `r` before the first quote: There is one subtle aspect to raw strings: a raw string may not end in an odd number of characters; see the FAQ entry for more information and workarounds. String literals can span multiple lines. One way is using triple-quotes: `'''...or...'''`. End-of-line characters are automatically included in the string, but it's possible to prevent this by adding `a` at the end of the line. In the following example, the initial newline is not included: Strings can be concatenated (glued together) with the `+` operator, and repeated with: Two or more string literals (i.e.

- the ones enclosed between quotes) next to each other are automatically concatenated. This feature is particularly useful when you want to break long strings: This only works with two literals though, not with variables or expressions: If you want to concatenate variables or a variable and a literal, use: `str + literal`. Strings can be indexed (subscripted), with the first character having index 0. There is no separate character type; a character is simply a string of size one: Indices may also be negative numbers, to start counting from the right: Note that since `-0` is the same as `0`, negative indices start from `-1`. In addition to indexing, slicing is also supported.

- While indexing is used to obtain individual characters, slicing allows you to obtain a substring: Slice indices have useful defaults; an omitted first index defaults to zero, an omitted second index defaults to the size of the string being sliced. Note how the start is always included, and the end always excluded. This makes sure that `s[i:i+1]` is always equal to `s[i]`: One way to remember how slices work is to think of the indices as pointing between characters, with the left edge of the first character numbered 0. Then the right edge of the last character of a string of `n` characters has index `n`, for example: The first row of numbers gives the position of the indices 0â€¦`n` in the string; the second row gives the corresponding negative indices.

- The slice from `i` to `j` consists of all characters between the edges labeled `i` and `j`, respectively. For non-negative indices, the length of a slice is the difference of the indices, if both are within bounds. For example, the length of `word[1:3]` is 2. Attempting to use an index that is too large will result in an error: However, out of range slice indexes are handled gracefully when used for slicing: Python strings cannot be changed â€” they are immutable.

- Therefore, assigning to an indexed position in the string results in an error: If you need a different string, you should create a new one: The built-in function `len()` returns the length of a string: Strings are examples of sequence types, and support the common operations supported by such types. Strings support a large number of methods for basic transformations and searching. String literals that have embedded expressions. Information about string formatting with `str.format()`.

- The old formatting operations invoked when strings are the left operand of the `+` operator are described in more detail here. Python knows a number of compound data types, used to group together other values. The most versatile is the list, which can be written as a list of comma-separated values (items) between square brackets. Lists might contain items of different types, but usually the items all have the same type.

- Like strings (and all other built-in sequence types), lists can be indexed and sliced: Lists also support operations like concatenation: Unlike strings, which are immutable, lists are a mutable type, i.e. it is possible to change their content: You can also add new items at the

end of the list, by using the `list.append()` method (we will see more about methods later): Simple assignment in Python never copies data. When you assign a list to a variable, the variable refers to the existing list. Any changes you make to the list through one variable will be seen through all other variables that refer to it.

- : All slice operations return a new list containing the requested elements. This means that the following slice returns a shallow copy of the list: Assignment to slices is also possible, and this can even change the size of the list or clear it entirely: The built-in function `len()` also applies to lists: It is possible to nest lists (create lists containing other lists), for example: 3.2. First Steps Towards Programming

Of course, we can use Python for more complicated tasks than adding two and two together. For instance, we can write an initial sub-sequence of the Fibonacci series as follows: This example introduces several new features. The first line contains a multiple assignment: the variables `a` and `b` simultaneously get the new values 0 and 1.

- On the last line this is used again, demonstrating that the expressions on the right-hand side are all evaluated first before any of the assignments take place. The right-hand side expressions are evaluated from the left to the right. The first line contains a multiple assignment: the variables `a` and `b` simultaneously get the new values 0 and 1. On the last line this is used again, demonstrating that the expressions on the right-hand side are all evaluated first before any of the assignments take place.

- The right-hand side expressions are evaluated from the left to the right. The `while` loop executes as long as the condition (here: `a < 10`) remains true. In Python, like in C, any non-zero integer value is true; zero is false. The condition may also be a string or list value, in fact any sequence; anything with a non-zero length is true, empty sequences are false.

- The test used in the example is a simple comparison. The standard comparison operators are written the same as in C: (less than), (greater than), (equal to), (less than or equal to), (greater than or equal to) and ! (not equal to). The `while` loop executes as long as the condition (here: `a < 10`) remains true.

- In Python, like in C, any non-zero integer value is true; zero is false. The condition may also be a string or list value, in fact any sequence; anything with a non-zero length is true, empty sequences are false. The test used in the example is a simple comparison. The standard comparison operators are written the same as in C: (less than), (greater than), (equal to), (less than or equal to), (greater than or equal to) and !

- (not equal to). The body of the loop is indented: indentation is Python's way of grouping statements. At the interactive prompt, you have to type a tab or space(s) for each indented line. In practice you will prepare more complicated input for Python with a text editor; all decent text editors have an auto-indent facility.

- When a compound statement is entered interactively, it must be followed by a blank line to indicate completion (since the parser cannot guess when you have typed the last line). Note that each line within a basic block must be indented by the same amount. The body of the loop is indented: indentation is Python's way of grouping statements. At the interactive prompt, you have to type a tab or space(s) for each indented line.

- In practice you will prepare more complicated input for Python with a text editor; all decent text editors have an auto-indent facility. When a compound statement is entered interactively, it must be followed by a blank line to indicate completion (since the parser cannot guess when you have typed the last line). Note that each line within a basic block must be indented by the same amount. The `print()` function writes the value of the argument(s) it is given.

- It differs from just writing the expression you want to write (as we did earlier in the calculator examples) in the way it handles multiple arguments, floating-point quantities, and strings. Strings are printed without quotes, and a space is inserted between items, so you can format things nicely, like this:
`i=256256`
`print('The value of i is',i)`
The value of i is 65536
The keyword argument `end` can be used to avoid the newline after the output, or end the output with a different string:
`a,b=0,1`
`while a<1000:...`
`print(a,end=',')...`
a,bb,ab...0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,
The `print()` function writes the value of the argument(s) it is given. It differs from just writing the expression you want to write (as we did earlier in the calculator examples) in the way it handles multiple arguments, floating-point quantities, and strings. Strings are printed without quotes, and a space is inserted between items, so you can format things nicely, like this: The keyword argument `end` can be used to avoid the newline after the output, or end the output with a different string: 3.

- An Informal Introduction to Python 3.1. Using Python as a Calculator 3.1.1. Numbers 3.1.2. Text 3.1.3.

- Lists 3.2. First Steps Towards Programming 3.1. Using Python as a Calculator 3.1.1. Numbers 3.1.2.

- Text 3.1.3. Lists 3.2. First Steps Towards Programming 3. An Informal Introduction to Python